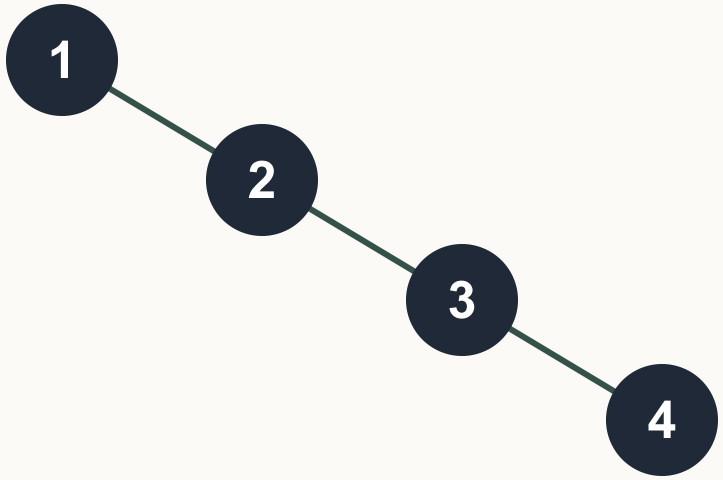
CSE 214: Data Structures

Lecture 7 — Balanced Search Trees

AVL trees · Rotations · Red-black trees

Why balance?

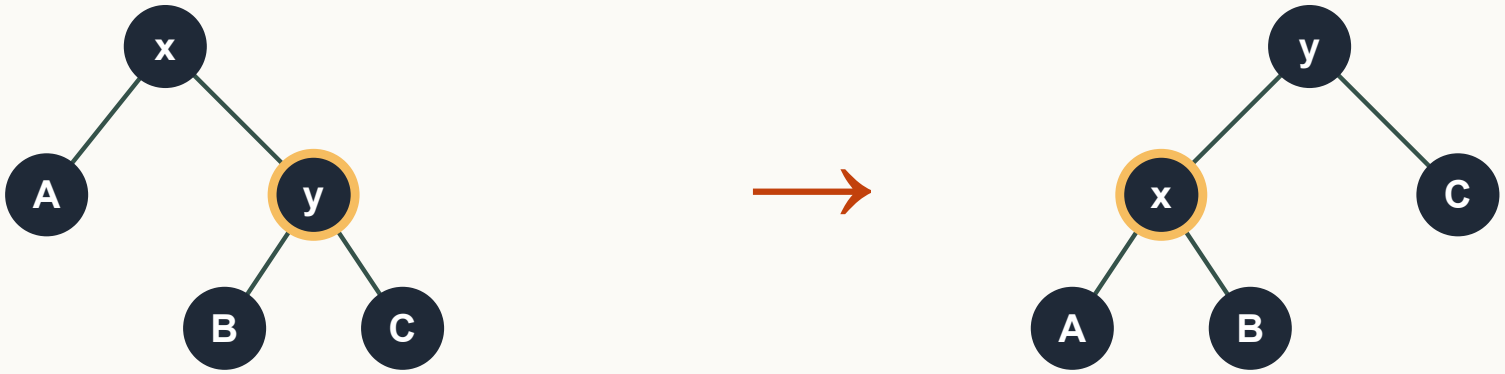
- A BST built from sorted input degenerates into a linked list
- Search, insert, delete become **O(n)** instead of **O(log n)**
- Balanced trees keep height ≤ c · log n after every update



Inserting 1, 2, 3, 4 in order gives height 3 for 4 keys.

Left rotation

Fix a right-heavy node **x** by making its right child **y** the new root of the subtree. In-order sequence is preserved.



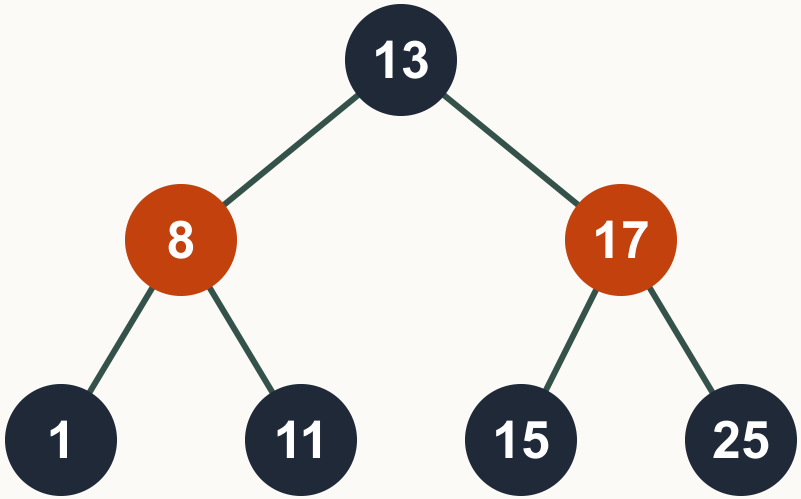
AVL insertion: the four cases

Imbalance	Shape	Fix
Left-Left	z.left.left heavy	Right rotate z
Right-Right	z.right.right heavy	Left rotate z
Left-Right	z.left.right heavy	Left rotate y, then right rotate z
Right-Left	z.right.left heavy	Right rotate y, then left rotate z

Balance factor = height(left) − height(right) ∈ {−1, 0, 1}

Red-black tree properties

1. Every node is red or black
2. The root is black
3. Red nodes have black children
4. Every root-to-null path has the same number of black nodes



Summary

- Rotations restore balance in O(1) each
- AVL: stricter balance, faster lookups
- Red-black: fewer rotations on insert/delete; used by Java's TreeMap
- **HW7:** implement AVL insert with all four rotation cases